

# MicroServices

## Study Guide

### Module 4

**Developing BookStore –V.1.2  
(Using RabbitMQ)**

**Author  
Srinivas Dande**





**CourseCube<sup>®</sup>**  
( Formerly Java Learning Center )



## Developing BookStore V-1.2

### Applying RabbitMQ in our BookStore App Eco-System

- RabbitMQ is the most widely deployed open source message broker.
- RabbitMQ is a message broker that implements AMQP (**Advanced Message Queing Protocol**).
- AMQP standardizes Messaging using Producers, Broker and Consumer.

### Step A : Using RabbitMQ in BookPriceMS-V1.2

- BookPriceMS is not sending any Async messages to any other MicroServices
- BookPriceMS is not receiving any Async messages from any other MicroServices
- So Rabbit MQ is not required for BookPriceMS

### Step B : Using RabbitMQ in BookSearchMS-V1.2

- **BookSearchMS** receives **BookInventoryInfo** message from **PlaceOrderMS** via RabbitMQ Server when Order is placed.
- **BookSearchMS** receives **BookRatingInfo** message from **UserRatingMS** via RabbitMQ Server when user gives the rating for any book.
- So we need to use Rabbit MQ in BookSearchMS.

### Working Steps:

#### 1) Add the following dependency to pom.xml

```
<dependency>  
<groupId>org.springframework.boot</groupId>  
<artifactId>spring-boot-starter-amqp</artifactId>  
</dependency>
```

#### 2) Add the following to application.properties

```
spring.rabbitmq.host=localhost  
spring.rabbitmq.port=5672  
spring.rabbitmq.username=guest  
spring.rabbitmq.password=guest
```



### 3) Define the required Beans for Exchange, Queue and Bindings for BookRating and BookInventory in JLCBookSearchConfig.java

```
public class JLCBookSearchConfig implements WebMvcConfigurer {

    // BookSearch Exchange
    @Bean(name = "myBookSearchExchange")
    Exchange createBookSearchExchange() {
        return ExchangeBuilder.topicExchange("mybook.search.exchange").build();
    }

    // BookRating Message
    //A. BookRating Queue
    @Bean(name = "myBookRatingsQueue")
    Queue createBookRatingsQueue() {
        return QueueBuilder.durable("mybook.ratings.queue").build();
    }

    //B. BookRating Binding
    @Bean
    Binding bookRatingBinding(Queue myBookRatingsQueue, TopicExchange
    myBookSearchExchange) {
        return BindingBuilder.bind(myBookRatingsQueue)
        .to(myBookSearchExchange)
        .with("mybook.ratings.key");
    }

    // BookInventory Message
    //A. BookInventory Queue
    @Bean(name = "myInventoryQueue")
    Queue createInventoryQueue() {
        return QueueBuilder.durable("myinventory.queue").build();
    }

    //B. BookInventory Binding
    @Bean
    Binding inventoryBinding(Queue myInventoryQueue, TopicExchange
    myBookSearchExchange) {
        return BindingBuilder.bind(myInventoryQueue)
        .to(myBookSearchExchange)
        .with("myinventory.key");
    }
}
```



4) Create the package called **com.jlcindia.rabbitmq** and place following two Serializable classes.

```
**BookRatingInfo.java
**BookInventoryInfo.java

public class BookRatingInfo implements Serializable{
    private Integer bookId;
    private double avgRating;
    private int numberOfSearches;
    ....
}

public class BookInventoryInfo implements Serializable {
    private Integer bookId;
    private int booksAvailable;
    ...
}
```

5) Remove the following two methods from **BookController.java**

```
public void updateBookRating(@RequestBody BookRating bookRating)
public void updateBookInventory(@RequestBody BookInventory bookInventory)
```

6) Remove the following two methods from **BookService.java**

```
public void updateBookRating(BookRating bookRating);
public void updateBookInventory(BookInventory bookInventory);
```

7) Update the methods of **BookServiceImpl.java**

```
@RabbitListener(queues = "mybook.ratings.queue")
public void updateBookRating(BookRatingInfo bookRatingInfo) {
    System.out.println("BookServiceImpl - updateBookRating");

    BookRating bookRating= new BookRating();
    bookRating.setBookId(bookRatingInfo.getBookId());
    bookRating.setAvgRating(bookRatingInfo.getAvgRating());
    bookRating.setNumberOfSearches(bookRatingInfo.getNumberOfSearches());

    bookRatingDAO.save(bookRating);
}
```

```
@RabbitListener(queues = "myinventory.queue")
public void updateBookInventory(BookInventoryInfo bookInventoryInfo) {
    System.out.println("BookServiceImpl - updateBookInventory");
```

```
    BookInventory bookInventory= new BookInventory();
    bookInventory.setBookId(bookInventoryInfo.getBookId());
    bookInventory.setBooksAvailable(bookInventoryInfo.getBooksAvailable());
```

```
    bookInventoryDAO.save(bookInventory);
}
```

## BookSearchMS – V-1.2: Files required

|                                     |                            |
|-------------------------------------|----------------------------|
| <b>1. BookController.java</b>       | <b>Updated in V1.2</b>     |
| 2. BookPriceInfo.java               | Same as V1.1               |
| 3. BookInfo.java                    | Same as V1.1               |
| <b>4. BookInventoryInfo.java</b>    | <b>Newly Added in V1.2</b> |
| <b>5. BookRatingInfo.java</b>       | <b>Newly Added in V1.2</b> |
| <b>6. BookService.java</b>          | <b>Updated in V1.2</b>     |
| <b>7. BookServiceImpl.java</b>      | <b>Updated in V1.2</b>     |
| 8. BookDAO.java                     | Same as V1.1               |
| 9. BookRatingDAO.java               | Same as V1.1               |
| 10. BookInventoryDAO.java           | Same as V1.1               |
| 11. Book.java                       | Same as V1.1               |
| 12. BookRating.java                 | Same as V1.1               |
| 13. BookInventory.java              | Same as V1.1               |
| <b>14. JLCBookSearchConfig.java</b> | <b>Updated in V1.2</b>     |
| 15. BookSearchApplication.java      | Same as V1.1               |
| <b>16. application.properties</b>   | <b>Updated in V1.2</b>     |
| <b>17. pom.xml</b>                  | <b>Updated in V1.2</b>     |

### 1.BookController.java

```
package com.jlcindia.booksearch;
```

```
import java.util.List;
```

```
import org.slf4j.Logger;
```

```
import org.slf4j.LoggerFactory;
```

```
import org.springframework.beans.factory.annotation.Autowired;
```

```
import org.springframework.web.bind.annotation.CrossOrigin;
```

```
import org.springframework.web.bind.annotation.GetMapping;
```



```
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;

import io.swagger.annotations.ApiOperation;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@CrossOrigin
@RestController
public class BookController {

    static Logger log = LoggerFactory.getLogger(BookController.class);

    @Autowired
    BookService bookService;

    @GetMapping("/mybooks/{author}/{category}")
    @ApiOperation(value = " getBooks", response = List.class, notes = "Returns List of
Books for given Author and Category")
    public List<Book> getBooks(@PathVariable String author, @PathVariable String category) {
        log.info("---BookController---getBooks()-----");
        System.out.println(author + "\t" + category);

        return bookService.getBooks(author, category);
    }

    @GetMapping("/mybook/{bookId}")
    @ApiOperation(value = " getBookById", response = BookInfo.class, notes = "Returns
BookInfo for given BID")
    public BookInfo getBookById(@PathVariable Integer bookId) {
        log.info("---BookController---getBookById()-----");

        return bookService.getBookInfo(bookId);
    }
}
```

#### **4.BookInventoryInfo.java**

```
package com.jlcindia.rabbitmq;

import java.io.Serializable;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
```



```
*/  
public class BookInventoryInfo implements Serializable {  
    private Integer bookId;  
    private int booksAvailable;  
  
    public BookInventoryInfo() {}  
    public BookInventoryInfo(Integer bookId, int booksAvailable) {  
        super();  
        this.bookId = bookId;  
        this.booksAvailable = booksAvailable;  
    }  
  
    //Setters and Getters  
}
```

### 5.BookRatingInfo.java

```
package com.jlcindia.rabbitmq;  
  
import java.io.Serializable;  
/*  
 * @Author : Srinivas Dande  
 * @company : Java Learning Center  
 */  
public class BookRatingInfo implements Serializable{  
    private Integer bookId;  
    private double avgRating;  
    private int numberOfSearches;  
  
    public BookRatingInfo() {}  
    public BookRatingInfo(Integer bookId, double avgRating, int numberOfSearches) {  
        super();  
        this.bookId = bookId;  
        this.avgRating = avgRating;  
        this.numberOfSearches = numberOfSearches;  
    }  
  
    //Setters and Getters  
}
```

### 6.BookService.java

```
package com.jlcindia.booksearch;  
  
import java.util.List;  
/*  
 * @Author : Srinivas Dande
```



```
* @company : Java Learning Center
```

```
**/
```

```
public interface BookService {  
    public List<Book> getBooks(String author,String category);  
    public BookInfo getBookInfo(Integer bookId);  
}
```

## 7. BookServiceImpl.java

```
package com.jlcindia.booksearch;
```

```
import java.util.ArrayList;
```

```
import java.util.List;
```

```
import java.util.Optional;
```

```
import org.slf4j.Logger;
```

```
import org.slf4j.LoggerFactory;
```

```
import org.springframework.amqp.rabbit.annotation.RabbitListener;
```

```
import org.springframework.beans.factory.annotation.Autowired;
```

```
import org.springframework.stereotype.Service;
```

```
import org.springframework.transaction.annotation.Transactional;
```

```
import org.springframework.web.client.RestTemplate;
```

```
import com.jlcindia.rabbitmq.BookInventoryInfo;
```

```
import com.jlcindia.rabbitmq.BookRatingInfo;
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @company : Java Learning Center
```

```
**/
```

```
@Service
```

```
@Transactional
```

```
public class BookServiceImpl implements BookService {
```

```
    static Logger log = LoggerFactory.getLogger(BookServiceImpl.class);
```

```
    @Autowired
```

```
    BookInventoryDAO bookInventoryDAO;
```

```
    @Autowired
```

```
    BookRatingDAO bookRatingDAO;
```

```
    @Autowired
```

```
    BookDAO bookDAO;
```



```
@Override
public List<Book> getBooks(String author, String category) {
    List<Book> mybooks = new ArrayList<>();

    if (author.equals("All Authors") && category.equals("All Categories")) {
        mybooks = bookDAO.findAll();
    } else if (author.equals("All Authors") && !category.equals("All Categories")) {
        mybooks = bookDAO.getBooksByCategory(category);
    } else if (!author.equals("All Authors") && category.equals("All Categories")) {
        mybooks = bookDAO.getBooksByAuthor(author);
    } else {
        mybooks = bookDAO.getBooksByAuthorAndCategory(author, category);
    }
    return mybooks;
}

@Override
public BookInfo getBookInfo(Integer bookId) {

    BookInfo bookInfo = new BookInfo();

    // 1. Book Details
    Optional<Book> myOpts = bookDAO.findById(bookId);
    if (myOpts.isPresent()) {
        Book book = myOpts.get();

        bookInfo.setBookId(book.getBookId());
        bookInfo.setBookName(book.getBookName());
        bookInfo.setAuthor(book.getAuthor());
        bookInfo.setPublications(book.getPublications());
        bookInfo.setCategory(book.getCategory());
    }

    // 2. Book Price Details (Invoking BookPrice-MS )
    RestTemplate bookPriceRest = new RestTemplate();
    String endpoint = "http://localhost:9000/bookPrice/" + bookId;
    BookPriceInfo bpInfo = bookPriceRest.getForObject(endpoint, BookPriceInfo.class);

    bookInfo.setPrice(bpInfo.getPrice());
    bookInfo.setOffer(bpInfo.getOffer());

    // 3. Book rating Details
    BookRating bookRating = bookRatingDAO.getBookRatingByBookId(bookId);
```



```
bookInfo.setAvgRating(bookRating.getAvgRating());
bookInfo.setNumberOfSearches(bookRating.getNumberOfSearches());

return bookInfo;
}

@RabbitListener(queues = "mybook.ratings.queue")
public void updateBookRating(BookRatingInfo bookRatingInfo) {
    System.out.println("BookServiceImpl - updateBookRating");

    BookRating bookRating= new BookRating();
    bookRating.setBookId(bookRatingInfo.getBookId());
    bookRating.setAvgRating(bookRatingInfo.getAvgRating());
    bookRating.setNumberOfSearches(bookRatingInfo.getNumberOfSearches());

    bookRatingDAO.save(bookRating);

}

@RabbitListener(queues = "myinventory.queue")
public void updateBookInventory(BookInventoryInfo bookInventoryInfo) {
    System.out.println("BookServiceImpl - updateBookInventory");

    BookInventory bookInventory= new BookInventory();
    bookInventory.setBookId(bookInventoryInfo.getBookId());
    bookInventory.setBooksAvailable(bookInventoryInfo.getBooksAvailable());

    bookInventoryDAO.save(bookInventory);

}
}
```

#### **14.JLCBookSearchConfig.java**

```
package com.jlcindia.booksearch;

import org.springframework.amqp.core.Binding;
import org.springframework.amqp.core.BindingBuilder;
import org.springframework.amqp.core.Exchange;
import org.springframework.amqp.core.ExchangeBuilder;
import org.springframework.amqp.core.Queue;
import org.springframework.amqp.core.QueueBuilder;
import org.springframework.amqp.core.TopicExchange;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;
```



```
import org.springframework.web.servlet.config.annotation.ResourceHandlerRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
```

```
import com.google.common.base.Predicates;
```

```
import springfox.documentation.builders.PathSelectors;
import springfox.documentation.builders.RequestHandlerSelectors;
import springfox.documentation.service.ApiInfo;
import springfox.documentation.service.Contact;
import springfox.documentation.spi.DocumentationType;
import springfox.documentation.spring.web.plugins.Docket;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
```

```
@SpringBootApplication
```

```
public class JLCBookSearchConfig implements WebMvcConfigurer {
```

```
// BookSearch Exchange
```

```
@Bean(name = "myBookSearchExchange")
```

```
Exchange createBookSearchExchange() {
```

```
return ExchangeBuilder.topicExchange("mybook.search.exchange").build();
```

```
}
```

```
// BookRating Message
```

```
//A. BookRating Queue
```

```
@Bean(name = "myBookRatingsQueue")
```

```
Queue createBookRatingsQueue() {
```

```
return QueueBuilder.durable("mybook.ratings.queue").build();
```

```
}
```

```
//B. BookRating Binding
```

```
@Bean
```

```
Binding bookRatingBinding(Queue myBookRatingsQueue, TopicExchange  
myBookSearchExchange) {
```

```
return BindingBuilder.bind(myBookRatingsQueue)
```

```
.to(myBookSearchExchange)
```

```
.with("mybook.ratings.key");
```

```
}
```

```
// BookInventory Message
```

```
//A. BookInventory Queue
```

```
@Bean(name = "myInventoryQueue")
```



```
Queue createInventoryQueue() {
return QueueBuilder.durable("myinventory.queue").build();
}

//B. BookInventory Binding
@Bean
Binding inventoryBinding(Queue myInventoryQueue, TopicExchange
myBookSearchExchange) {
return BindingBuilder.bind(myInventoryQueue)
.to(myBookSearchExchange)
.with("myinventory.key");
}

private ApiInfo getApiDetails() {
return new ApiInfo("JLC BookStore- API", "BookSearchMS- API - part of BookStore", "1.0",
"Free to use ",
new Contact("Srinivas Dande", "https://www.jlcindia.com", "sri@jlcindia.com"), "API Under
Free Licence",
"https://www.jlcindia.com");
}

@Bean
public Docket api() {
return new Docket(DocumentationType.SWAGGER_2).select().paths(PathSelectors.any())
.apis(Predicates.not(RequestHandlerSelectors.basePackage("org.springframework.boot")))
.build()
.apiInfo(getApiDetails());
}

@Override
public void addResourceHandlers(ResourceHandlerRegistry registry) {
//enabling swagger-ui
registry.addResourceHandler("swagger-ui.html").addResourceLocations("classpath:/META-
INF/resources/");

registry.addResourceHandler("/webjars/**").addResourceLocations("classpath:/META-
INF/resources/webjars/");
}
}
```

## 16. application.properties

spring.application.name=MyBookSearchMS

server.port=8000



```
spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest

spring.boot.admin.client.url=http://localhost:5555

management.endpoints.web.exposure.include=*
management.endpoint.health.show-details=always

spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/jlcbooksdb
spring.datasource.username=root
spring.datasource.password=srinivas

spring.datasource.hikari.connectionTimeout=20000
spring.datasource.hikari.maximumPoolSize=5

logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %msg%n

spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update

# INFO ENDPOINT CONFIGURATION
info.app.name=@project.name@
info.app.description=@project.description@
info.app.version=@project.version@
info.app.encoding=@project.build.sourceEncoding@
info.app.java.version=@java.version@
info.app.developer=Srinivas Dande
info.app.company=Java Learning Center
```

## **17. pom.xml**

```
<project ...>
<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.booksearch</groupId>
<artifactId>BookSearchMS</artifactId>
<version>1.2</version>
<packaging>jar</packaging>
```



```
<name>MyBookSearchMS</name>
<url>https://www.jlcindia.com</url>
<description>This is BookSeach MS developed by Srinivas Dande</description>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.5.3</version>
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-amqp</artifactId>
</dependency>

<dependency>
<groupId>de.codecentric</groupId>
<artifactId>spring-boot-admin-starter-client</artifactId>
<version>2.4.3</version>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
<version>5.1.6</version>
</dependency>
```



```
<dependency>
<groupId>io.springfox</groupId>
<artifactId>springfox-swagger2</artifactId>
<version>2.4.0</version>
</dependency>

<dependency>
<groupId>io.springfox</groupId>
<artifactId>springfox-swagger-ui</artifactId>
<version>2.4.0</version>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-devtools</artifactId>
</dependency>
</dependencies>

<build>
<finalName>MyBookSearchMS</finalName>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```

| MicroService Name | Rest Endpoints                                    |
|-------------------|---------------------------------------------------|
| MyBookSearchMS    | http://localhost:8000/mybooks/{author}/{category} |
| MyBookSearchMS    | http://localhost:8000/mybook/{bookId}             |

## Step C : Using RabbitMQ in PlaceOrderMS-V.1.2

### Using RabbitMQ in PlaceOrderMS-V.1.2

- PlaceOrderMS sends BookInventoryInfo message to BookSearchMS via Rabbit MQ Asynchronously
- PlaceOrderMS Receives OrderFullInfo message from BookStoreWeb via Rabbit MQ Asynchronously
- So we need to use RabbitMQ in PlaceOrderMS.

### Working Steps:

#### 1) Add the following dependency to pom.xml

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-amqp</artifactId>
</dependency>
```

#### 2) Add the following to application.properties

```
spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest
```

#### 3) Add the Queue and Exchange for Inventory in JLCPlaceOrderConfig.java

```
public class JLCPlaceOrderConfig implements WebMvcConfigurer {

    // Order Message
    //A. Order Exchange
    @Bean(name="myOrderExchange")
    Exchange createOrderExchange() {
        return ExchangeBuilder.topicExchange("myorder.exchange").build();
    }

    //B. Order Queue
    @Bean(name="myOrderQueue")
    Queue createOrderQueue() {
        return QueueBuilder.durable("myorder.queue").build();
    }
}
```



```
//C. Order Binding
```

```
@Bean
```

```
Binding orderBinding(Queue myOrderQueue, TopicExchange
```

```
myOrderExchange) {
```

```
return BindingBuilder.bind(myOrderQueue)
```

```
.to(myOrderExchange).with("myorder.key");
```

```
}
```

```
}
```

- 4) Create the package called **com.jlcindia.rabbitmq** and place the following 4 **Serializable** classes.

```
***OrderInfo.java
```

```
***OrderItemInfo.java
```

```
***OrderFullInfo.java
```

```
***BookInventoryInfo.java
```

```
class OrderInfo implements Serializable{
```

```
    private Integer orderId;
```

```
    private String orderDate;
```

```
    private String userId;
```

```
    private int totalQty;
```

```
    private double totalCost;
```

```
    private String status;
```

```
    ...
```

```
}
```

```
class OrderItemInfo implements Serializable {
```

```
    private Integer orderItemId;
```

```
    private Integer orderId;
```

```
    private Integer bookId;
```

```
    private int qty;
```

```
    private double cost;
```

```
    ...
```

```
}
```

```
class OrderFullInfo implements Serializable {
```

```
    private Order order;
```

```
    private List<OrderItem> itemsList;
```

```
    ...
```

```
}
```



```
class BookInventoryInfo implements Serializable {  
    private Integer bookId;  
    private int booksAvailable;  
    ...  
}
```

**5) Remove the following in OrderController**

```
@PutMapping("/placeOrder")  
public void placeOrder(@RequestBody OrderInfo orderInfo){  
    ...  
}
```

**6) Remove the following method from OrderService interface**

```
public void placeOrder(OrderInfo orderInfo) ;
```

**7) Update the methods of OrderServiceImpl**

```
@Autowired  
private RabbitTemplate rabbitTemplate;  
  
@RabbitListener(queues = "myorder.queue")  
public void placeOrder(OrderFullInfo orderFullInfo) {  
  
    log.info("--3.OrderServiceImpl---placeOrder()-----");  
  
    //4 Tasks To DO When You Place Order  
  
    // Task1: Insert Order - 1  
    .....  
  
    // Task2: Insert OrderItems - N  
    .....  
  
    // Task3: Update Local BookInventory - N  
    // Task4: Update BookSearchMS BookInventory - N  
  
    for(OrderItemInfo orderItemInfo:itemsListInfo) {  
  
        Integer bookId=orderItemInfo.getBookId();  
        BookInventory mybookInventory = bookInventoryDAO.findById(bookId).get();  
        Integer currentStock=mybookInventory.getBooksAvailable();  
        currentStock = currentStock - orderItemInfo.getQty();  
        mybookInventory.setBooksAvailable(currentStock);  
    }  
}
```



```
// Update Local Inventory
```

```
bookInventoryDAO.save(mybookInventory);
```

```
//Update Inventory of BookSearchMS by Sending Message
```

```
BookInventoryInfo bookInventoryInfo=new BookInventoryInfo();
```

```
bookInventoryInfo.setBookId(mybookInventory.getBookId());
```

```
bookInventoryInfo.setBooksAvailable(mybookInventory.getBooksAvailable());
```

```
rabbitTemplate.convertAndSend("mybook.search.exchange","myinventory.key",  
bookInventoryInfo);
```

```
} //end of for loop
```

```
}
```

### PlaceOrderMS – V-1.2: Files required

|                                |                             |
|--------------------------------|-----------------------------|
| 1. OrderController.java        | <b>Updated in V1.2</b>      |
| 2. OrderFullInfo.java          | <b>Newly Added in V.1.2</b> |
| 3. OrderInfo.java              | <b>Newly Added in V.1.2</b> |
| 4. OrderItemInfo.java          | <b>Newly Added in V.1.2</b> |
| 5. BookInventoryInfo.java      | <b>Newly Added in V.1.2</b> |
| 6. OrderService.java           | <b>Updated in V1.2</b>      |
| 7. OrderServiceImpl.java       | <b>Updated in V1.2</b>      |
| 8. OrderDAO.java               | Same as V1.1                |
| 9. OrderItemDAO.java           | Same as V1.1                |
| 10. BookInventoryDAO.java      | Same as V1.1                |
| 11. Order.java                 | Same as V1.1                |
| 12. OrderItem.java             | Same as V1.1                |
| 13. BookInventory.java         | Same as V1.1                |
| 14. JLCPlaceOrderConfig.java   | <b>Updated in V1.2</b>      |
| 15. PlaceOrderApplication.java | Same as V1.0                |
| 16.application.properties      | <b>Updated in V1.2</b>      |
| 17.pom.xml                     | <b>Updated in V1.2</b>      |



### 1.OrderController.java

```
package com.jlcindia.placeorder;

import java.util.List;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.CrossOrigin;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;

import io.swagger.annotations.ApiOperation;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@CrossOrigin
@RestController
public class OrderController {
    static Logger log = LoggerFactory.getLogger(OrderController.class);

    @Autowired
    OrderService orderService;

    @GetMapping("/myorders/{userId}")
    @ApiOperation(value = " getOrdersByUserId", response = List.class, notes = "Return Orders belongs User")
    public List<Order> getOrdersByUserId(@PathVariable String userId){
        log.info("---OrderController---getOrdersByUserId()-----");
        List<Order> myorders= orderService.getOrdersByUserId(userId);
        return myorders;
    }
}
```

### 2.OrderFullInfo.java

```
package com.jlcindia.rabbitmq;

import java.io.Serializable;
import java.util.List;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
```



```
public class OrderFullInfo implements Serializable {  
  
    private OrderInfo order;  
    private List<OrderItemInfo> itemsList;  
  
        //Constructors  
        //Setters and Getters  
}
```

### 3. OrderInfo.java

```
package com.jlcindia.rabbitmq;  
  
import java.io.Serializable;  
/*  
 * @Author : Srinivas Dande  
 * @company : Java Learning Center  
 */  
public class OrderInfo implements Serializable{  
  
    private Integer orderId;  
    private String orderDate;  
    private String userId;  
    private int totalQty;  
    private double totalCost;  
    private String status;  
  
        //Constructors  
        //Setters and Getters  
}
```

### 3. OrderItemInfo.java

```
package com.jlcindia.rabbitmq;  
  
import java.io.Serializable;  
/*  
 * @Author : Srinivas Dande  
 * @company : Java Learning Center  
 */  
public class OrderItemInfo implements Serializable {  
  
    private Integer orderItemId;  
    private Integer orderId;
```



```
private Integer bookId;  
private int qty;  
private double cost;  
  
//Constructors  
//Setters and Getters  
  
}
```

### 5. BookInventoryInfo.java

```
package com.jlcindia.rabbitmq;  
  
import java.io.Serializable;  
/*  
 * @Author : Srinivas Dande  
 * @company : Java Learning Center  
 */  
public class BookInventoryInfo implements Serializable {  
    private Integer bookId;  
    private int booksAvailable;  
  
    //Constructors  
    //Setters and Getters  
    //toString  
  
}
```

### 6. OrderService.java

```
package com.jlcindia.placeorder;  
  
import java.util.List;  
/*  
 * @Author : Srinivas Dande  
 * @company : Java Learning Center  
 */  
public interface OrderService {  
    public List<Order> getOrdersByUserId(String userId);  
}
```



## 7. OrderServiceImpl.java

```
package com.jlcindia.placeorder;

import java.util.List;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.amqp.rabbit.annotation.RabbitListener;
import org.springframework.amqp.rabbit.core.RabbitTemplate;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import com.jlcindia.rabbitmq.BookInventoryInfo;
import com.jlcindia.rabbitmq.OrderFullInfo;
import com.jlcindia.rabbitmq.OrderInfo;
import com.jlcindia.rabbitmq.OrderItemInfo;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@Service
@Transactional
public class OrderServiceImpl implements OrderService{

static Logger log=LoggerFactory.getLogger(OrderServiceImpl.class);

    @Autowired
    OrderDAO orderDAO;

    @Autowired
    OrderItemDAO orderItemDAO;

    @Autowired
    BookInventoryDAO bookInventoryDAO;

    @Autowired
    private RabbitTemplate rabbitTemplate;

    @RabbitListener(queues = "myorder.queue")
    public void placeOrder(OrderFullInfo orderFullInfo) {
        log.info("--3.OrderServiceImpl---placeOrder()-----");

        //4 Tasks To DO When You Place Order
    }
}
```



**// Task1: Insert Order - 1**

```
OrderInfo orderInfo=orderFullInfo.getOrder();
```

```
Order myorder=new Order(orderInfo.getOrderDate(), orderInfo.getUserId(),  
orderInfo.getTotalQty(), orderInfo.getTotalCost(), orderInfo.getStatus());  
myorder=orderDAO.save(myorder);//1002  
int orderId=myorder.getOrderId();
```

**// Task2: Insert OrderItems - N**

```
List<OrderItemInfo> itemsListInfo=orderFullInfo.getItemsList();  
for(OrderItemInfo orderItemInfo:itemsListInfo) {  
OrderItem myorderItem=new OrderItem(orderId, orderItemInfo.getBookId(),  
orderItemInfo.getQty(), orderItemInfo.getCost());  
orderItemDAO.save(myorderItem);  
}
```

**// Task3: Update Local BookInventory - N**

**// Task4: Update BookSearchMS BookInventory - N**

```
//101,102,103
```

```
for(OrderItemInfo orderItemInfo:itemsListInfo) {
```

```
Integer bookId=orderItemInfo.getBookId(); //103
```

```
BookInventory mybookInventory = bookInventoryDAO.findById(bookId).get();
```

```
Integer currentStock=mybookInventory.getBooksAvailable();  
currentStock = currentStock - orderItemInfo.getQty();  
mybookInventory.setBooksAvailable(currentStock);
```

**// Update Local Inventory**

```
bookInventoryDAO.save(mybookInventory);
```

**//Update Inventory of BookSearchMS by Sending Message**

```
BookInventoryInfo bookInventoryInfo=new BookInventoryInfo();  
bookInventoryInfo.setBookId(mybookInventory.getBookId());  
bookInventoryInfo.setBooksAvailable(mybookInventory.getBooksAvailable());
```

```
rabbitTemplate.convertAndSend("mybook.search.exchange","myinventory.key",  
bookInventoryInfo);
```

```
} //end of for loop  
}
```



```
@Override
public List<Order> getOrdersByUserId(String userId) {
    log.info("---OrderServiceImpl---getOrdersByUserId()-----");
    List<Order> orderList=orderDAO.findOrdersByUserId(userId);
    return orderList;
}

}
```

#### 14.JLCPlaceOrderConfig.java

```
package com.jlcindia.placeorder;

import org.springframework.amqp.core.Binding;
import org.springframework.amqp.core.BindingBuilder;
import org.springframework.amqp.core.Exchange;
import org.springframework.amqp.core.ExchangeBuilder;
import org.springframework.amqp.core.Queue;
import org.springframework.amqp.core.QueueBuilder;
import org.springframework.amqp.core.TopicExchange;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;
import org.springframework.web.servlet.config.annotation.ResourceHandlerRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

import com.google.common.base.Predicates;

import springfox.documentation.builders.PathSelectors;
import springfox.documentation.builders.RequestHandlerSelectors;
import springfox.documentation.service.ApiInfo;
import springfox.documentation.service.Contact;
import springfox.documentation.spi.DocumentationType;
import springfox.documentation.spring.web.plugins.Docket;

@SpringBootApplication
public class JLCPlaceOrderConfig implements WebMvcConfigurer {

    // Order Message
    //A. Order Exchange
    @Bean(name="myOrderExchange")
    Exchange createOrderExchange() {
        return ExchangeBuilder.topicExchange("myorder.exchange").build();
    }
}
```



#### //B. Order Queue

```
@Bean(name="myOrderQueue")
Queue createOrderQueue() {
    return QueueBuilder.durable("myorder.queue").build();
}
```

#### //C. Order Binding

```
@Bean
Binding orderBinding(Queue myOrderQueue, TopicExchange myOrderExchange) {
    return BindingBuilder.bind(myOrderQueue)
        .to(myOrderExchange).with("myorder.key");
}
```

```
private ApiInfo getApiDetails() {
    return new ApiInfo("JLC BookStore- API",
        "PlaceOrderMS- API - part of BookStore", "1.0",
        "Free to use ",
        new Contact("Srinivas Dande", "https://www.jlcindia.com", "sri@jlcindia.com"),
        "API Under Free Licence",
        "https://www.jlcindia.com");
}
```

```
@Bean
public Docket api() {
    return new Docket(DocumentationType.SWAGGER_2).select().paths(PathSelectors.any())
        .apis(Predicates.not(RequestHandlerSelectors.basePackage("org.springframework.boot")))
        .build()
        .apiInfo(getApiDetails());
}
```

```
@Override
public void addResourceHandlers(ResourceHandlerRegistry registry) {
    //enabling swagger-ui
    registry.addResourceHandler("swagger-ui.html").addResourceLocations("classpath:/META-INF/resources/");

    registry.addResourceHandler("/webjars/**").addResourceLocations("classpath:/META-INF/resources/webjars/");
}
}
```



## **16. application.properties**

**spring.application.name=MyPlaceOrderMS**

**server.port=7000**

**spring.boot.admin.client.url=http://localhost:5555**

**spring.rabbitmq.host=localhost**

**spring.rabbitmq.port=5672**

**spring.rabbitmq.username=guest**

**spring.rabbitmq.password=guest**

**management.endpoints.web.exposure.include=\***

**management.endpoint.health.show-details=always**

**spring.datasource.driver-class-name=com.mysql.jdbc.Driver**

**spring.datasource.url=jdbc:mysql://localhost:3306/jlcordersdb**

**spring.datasource.username=root**

**spring.datasource.password=srinivas**

**spring.datasource.hikari.connectionTimeout=20000**

**spring.datasource.hikari.maximumPoolSize=5**

**logging.level.root=INFO**

**logging.pattern.console=%-5level %logger{36} - %msg%n**

**spring.jpa.show-sql=true**

**spring.jpa.hibernate.ddl-auto=update**

**# INFO ENDPOINT CONFIGURATION**

**info.app.name=@project.name@**

**info.app.description=@project.description@**

**info.app.version=@project.version@**

**info.app.encoding=@project.build.sourceEncoding@**

**info.app.java.version=@java.version@**

**info.app.developer=Srinivas Dande**

**info.app.company=Java Learning Center**



## 17. pom.xml

---

```
<project ...>

<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.placeorder</groupId>
<artifactId>PlaceOrderMS</artifactId>
<version>1.2</version>
<packaging>jar</packaging>

<name>MyPlaceOrderMS</name>
<url>https://www.jlcindia.com</url>
<description>This is PlaceOrderMS developed by Srinivas Dande</description>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.5.3</version>
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-amqp</artifactId>
</dependency>

<dependency>
<groupId>de.codecentric</groupId>
<artifactId>spring-boot-admin-starter-client</artifactId>
<version>2.4.3</version>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>
```



```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
<version>5.1.6</version>
</dependency>

<dependency>
<groupId>io.springfox</groupId>
<artifactId>springfox-swagger2</artifactId>
<version>2.4.0</version>
</dependency>

<dependency>
<groupId>io.springfox</groupId>
<artifactId>springfox-swagger-ui</artifactId>
<version>2.4.0</version>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-devtools</artifactId>
</dependency>
</dependencies>

<build>
<finalName>MyPlaceOrderMS</finalName>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```

| MicroService Name | Rest Endpoints                         |
|-------------------|----------------------------------------|
| MyPlaceOrderMS    | http://localhost:7000/myoders/{userId} |



## Step D : Using RabbitMQ in UserRatingMS-V.1.2

### Using RabbitMQ in UserRatingMS-V.1.2

- UserRatingMS receives UserRatingInfo message from BookSearchWeb via RabbitMQ
- UserRatingMS sends BookRatingInfo message to BookSearchMS via RabbitMQ
- So we need to use Rabbit MQ in UserRatingMS.

### Working Steps:

#### 1) Add the following dependency to pom.xml

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-amqp</artifactId>
</dependency>
```

#### 2) Add the following to application.properties

```
spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest
```

#### 3) Add the Queue and Exchange for Ratings in JLCUserRatingConfig.java

```
class JLCUserRatingConfig implements WebMvcConfigurer {

    //UserRating Message
    //A.UserRating Exchange
    @Bean(name = "myUserRatingsExchange")
    Exchange createUserRatingsExchange() {
        return ExchangeBuilder.topicExchange("myuser.ratings.exchange").build();
    }

    //B.UserRating Queue
    @Bean(name = "myUserRatingsQueue")
    Queue createUserRatingsQueue() {
        return QueueBuilder.durable("myuser.ratings.queue").build();
    }
}
```



```
//C.UserRating Binding
@Bean
Binding userRatingBinding(Queue myUserRatingsQueue, TopicExchange
myUserRatingsExchange) {
return BindingBuilder.bind(myUserRatingsQueue)
.to(myUserRatingsExchange).with("myuser.ratings.key");
}
}
```

- 4) Create the package called **com.jlcindia.rabbitmq** and Place the following two **Serializable** classes.

```
***BookRatingInfo.java
***UserRatingInfo.java

class BookRatingInfo implements Serializable{
    private Integer bookId;
    private double avgRating;
    private int numberOfSearches;
    ...
}

class UserRatingInfo implements Serializable{

    private Integer ratingId;
    private String userId;
    private Integer bookId;
    private double rating;
    private String review;
    ...
}
```

- 5) Remove the following method from **RatingController**

```
@PutMapping("/addUserRating")
public void addUserRating(@RequestBody UserRating userRating)
```

- 6) Remove the following method from **RatingService**

```
public void addUserRating(UserRating userRating);
```



## 7) Update the methods of RatingServiceImpl.java

```
@Autowired
private RabbitTemplate rabbitTemplate;

....

//4.Update BookRating in BookSearchMS By Sending Message to RabbitMQ
BookRatingInfo bookRatingInfo=new BookRatingInfo();

bookRatingInfo.setBookId(bookRating.getBookId());
bookRatingInfo.setAvgRating(bookRating.getAvgRating());
bookRatingInfo.setNumberOfSearches(bookRating.getNumberOfSearches());

rabbitTemplate.convertAndSend("mybook.search.exchange","mybook.rating
s.key", bookRatingInfo);
```

### UserRatingMS – V-1.2: Files required

|                                  |                             |
|----------------------------------|-----------------------------|
| 1. RatingController.java         | <b>Updated in V1.2</b>      |
| <b>2. UserRatingInfo.java</b>    | <b>Newly Added in V.1.2</b> |
| <b>3. BookRatingInfo.java</b>    | <b>Newly Added in V.1.2</b> |
| 4. RatingService.java            | <b>Updated in V1.2</b>      |
| 5. RatingServiceImpl.java        | <b>Updated in V1.2</b>      |
| 6. UserRatingDAO.java            | Same as V1.1                |
| 7. BookRatingDAO.java            | Same as V1.1                |
| 8. UserRating.java               | Same as V1.1                |
| 9. BookRating.java               | Same as V1.1                |
| 10. JLCUserRatingConfig.java     | <b>Updated in V1.2</b>      |
| 11. UserRatingApplication.java   | Same as V1.1                |
| <b>12.application.properties</b> | <b>Updated in V1.2</b>      |
| <b>13.pom.xml</b>                | <b>Updated in V1.2</b>      |



### 1. RatingController.java

```
package com.jlcindia.userratings;

import java.util.List;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.CrossOrigin;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@CrossOrigin
@RestController
public class RatingController {

    static Logger log = LoggerFactory.getLogger(RatingController.class);

    @Autowired
    RatingService ratingService;

    @GetMapping("/userRatings/{userId}")
    public List<UserRating> getUserRatingByUserId(@PathVariable String userId) {
        log.info("---RatingController---getUserRatingByUserId()-----");

        return ratingService.getUserRatingByUserId(userId);
    }
}
```

### 2. UserRatingInfo.java

```
package com.jlcindia.rabbitmq;

import java.io.Serializable;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
public class UserRatingInfo implements Serializable{
```



```
private Integer ratingId;  
private String userId;  
private Integer bookId;  
private double rating;  
private String review;
```

```
//Constructors  
//Setters and Getters  
//toString
```

```
}
```

### 3.BookRatingInfo.java

```
package com.jlcindia.rabbitmq;
```

```
import java.io.Serializable;
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @company : Java Learning Center
```

```
**/
```

```
public class BookRatingInfo implements Serializable{
```

```
    private Integer bookId;
```

```
    private double avgRating;
```

```
    private int numberOfSearches;
```

```
    //Constructors
```

```
    //Setters and Getters
```

```
}
```

### 4.RatingService.java

```
package com.jlcindia.userratings;
```

```
import java.util.List;
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @company : Java Learning Center
```

```
**/
```

```
public interface RatingService {
```

```
    public List<UserRating> getUserRatingByBookId(Integer bookId);
```

```
    public List<UserRating> getUserRatingByUserId(String userId);
```

```
    public BookRating getBookRatingByBookId(Integer bookId);
```

```
}
```



## 5. RatingServiceImpl.java

```
package com.jlcindia.userratings;

import java.util.List;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.amqp.rabbit.annotation.RabbitListener;
import org.springframework.amqp.rabbit.core.RabbitTemplate;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import com.jlcindia.rabbitmq.BookRatingInfo;
import com.jlcindia.rabbitmq.UserRatingInfo;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

@Service
@Transactional
public class RatingServiceImpl implements RatingService{

    static Logger log=LoggerFactory.getLogger(RatingServiceImpl.class);

    @Autowired
    BookRatingDAO bookRatingDAO;

    @Autowired
    UserRatingDAO userRatingDAO;

    @Autowired
    private RabbitTemplate rabbitTemplate;

    @RabbitListener(queues = "myuser.ratings.queue")
    public void addUserRating(UserRatingInfo urInfo) {
        log.info(" 3. RatingServiceImpl - addUserRating");

        UserRating userRating=new UserRating(urInfo.getBookId(), urInfo.getUserId(),
        urInfo.getRating(), urInfo.getReview());
        //1.Add the User Rating
        userRatingDAO.save(userRating); //BookID
```



**//2.Calculate the Avg rating for BookId**

```
int bookId=userRating.getBookId(); //101
List<UserRating> ratingList=this.getUserRatingByBookId(bookId);
```

```
double sumRating=0.0;
for(UserRating ur : ratingList) {
    sumRating=sumRating+ur.getRating();
}
double avgRating=sumRating/ratingList.size();
```

**//3.Update BookRating in UserRatingMS (Local)**

```
BookRating bookRating=bookRatingDAO.findById(bookId).get();
bookRating.setAvgRating(avgRating);
bookRatingDAO.save(bookRating);
```

**//4.Update BookRating in BookSearchMS By Sending Message to RabbitMQ**

```
BookRatingInfo bookRatingInfo=new BookRatingInfo();
bookRatingInfo.setBookId(bookRating.getBookId());
bookRatingInfo.setAvgRating(bookRating.getAvgRating());
bookRatingInfo.setNumberOfSearches(bookRating.getNumberOfSearches());
```

```
rabbitTemplate.convertAndSend("mybook.search.exchange","mybook.ratings.key",
bookRatingInfo);
}
```

```
@Override
public List<UserRating> getUserRatingByBookId(Integer bookId) {
    return userRatingDAO.findUserRatingByBookId(bookId);
}
```

```
@Override
public List<UserRating> getUserRatingByUserId(String userId) {
    return userRatingDAO.findUserRatingByUserId(userId);
}
```

```
@Override
public BookRating getBookRatingByBookId(Integer bookId) {
    BookRating bookRating=bookRatingDAO.findBookRatingByBookId(bookId);
    return bookRating;
}
}
```



### 10.JLCUserRatingConfig.java

```
package com.jlcindia.userratings;

import org.springframework.amqp.core.Binding;
import org.springframework.amqp.core.BindingBuilder;
import org.springframework.amqp.core.Exchange;
import org.springframework.amqp.core.ExchangeBuilder;
import org.springframework.amqp.core.Queue;
import org.springframework.amqp.core.QueueBuilder;
import org.springframework.amqp.core.TopicExchange;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;
import org.springframework.web.servlet.config.annotation.ResourceHandlerRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

import com.google.common.base.Predicates;

import springfox.documentation.builders.PathSelectors;
import springfox.documentation.builders.RequestHandlerSelectors;
import springfox.documentation.service.ApiInfo;
import springfox.documentation.service.Contact;
import springfox.documentation.spi.DocumentationType;
import springfox.documentation.spring.web.plugins.Docket;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@SpringBootApplication
public class JLCUserRatingConfig implements WebMvcConfigurer {

    //UserRating Message
    //A.UserRating Exchange
    @Bean(name = "myUserRatingsExchange")
    Exchange createUserRatingsExchange() {
        return ExchangeBuilder.topicExchange("myuser.ratings.exchange").build();
    }

    //B.UserRating Queue
    @Bean(name = "myUserRatingsQueue")
    Queue createUserRatingsQueue() {
        return QueueBuilder.durable("myuser.ratings.queue").build();
    }
}
```



### //C.UserRating Binding

@Bean

```
Binding userRatingBinding(Queue myUserRatingsQueue, TopicExchange  
myUserRatingsExchange) {  
    return BindingBuilder.bind(myUserRatingsQueue)  
        .to(myUserRatingsExchange).with("myuser.ratings.key");  
}
```

```
private ApiInfo getApiDetails() {  
    return new ApiInfo("JLC BookStore- API",  
        "UserRatingMS- API - part of BookStore", "1.0",  
        "Free to use ",  
        new Contact("Srinivas Dande", "https://www.jlcindia.com", "sri@jlcindia.com"),  
        "API Under Free Licence",  
        "https://www.jlcindia.com");  
}
```

@Bean

```
public Docket api() {  
    return new Docket(DocumentationType.SWAGGER_2).select().paths(PathSelectors.any())  
        .apis(Predicates.not(RequestHandlerSelectors.basePackage("org.springframework.boot")))  
        .build()  
        .apiInfo(getApiDetails());  
}
```

@Override

```
public void addResourceHandlers(ResourceHandlerRegistry registry) {  
    //enabling swagger-ui  
    registry.addResourceHandler("swagger-ui.html").addResourceLocations("classpath:/META-  
INF/resources/");  
  
    registry.addResourceHandler("/webjars/**").addResourceLocations("classpath:/META-  
INF/resources/webjars/");  
}  
  
}
```

### 12. application.properties

spring.application.name=**MyUserRatingMS**

server.port=**6500**

spring.boot.admin.client.url=**http://localhost:5555**



```
spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest

management.endpoints.web.exposure.include=*
management.endpoint.health.show-details=always

spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/jlcratingsdb
spring.datasource.username=root
spring.datasource.password=srinivas

spring.datasource.hikari.connectionTimeout=20000
spring.datasource.hikari.maximumPoolSize=5

logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %msg%n

spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update

# INFO ENDPOINT CONFIGURATION
info.app.name=@project.name@
info.app.description=@project.description@
info.app.version=@project.version@
info.app.encoding=@project.build.sourceEncoding@
info.app.java.version=@java.version@
info.app.developer=Srinivas Dande
info.app.company=Java Learning Center
```

### **13. pom.xml**

```
<project ...>
<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.userrating</groupId>
<artifactId>UserRatingMS</artifactId>
<version>1.2</version>
<packaging>jar</packaging>

<name>MyUserRatingMS</name>
<url>https://www.jlcindia.com</url>
<description>This is UserRatingMS developed by Srinivas Dande</description>
```



```
<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.5.3</version>
</parent>

<dependencies>
<b><dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-amqp</artifactId>
</dependency>

<dependency>
<groupId>de.codecentric</groupId>
<artifactId>spring-boot-admin-starter-client</artifactId>
<version>2.4.3</version>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
<version>5.1.6</version>
</dependency>

<dependency>
<groupId>io.springfox</groupId>
<artifactId>springfox-swagger2</artifactId>
<version>2.4.0</version>
</dependency>
```



```
<dependency>
<groupId>io.springfox</groupId>
<artifactId>springfox-swagger-ui</artifactId>
<version>2.4.0</version>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-devtools</artifactId>
</dependency>

</dependencies>

<build>
<finalName>MyUserRatingMS</finalName>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```

| MicroService Name | Rest Endpoints                             |
|-------------------|--------------------------------------------|
| MyUserRatingMS    | http://localhost:6500/userRatings/{userId} |



## Step E : Using RabbitMQ in BookStoreWeb-V1.2

### Using RabbitMQ in BookStoreWeb-V.1.2

- BookStoreWeb sends the message to PlaceOrderMS via Rabbit MQ Asynchronously
- BookStoreWeb sends the message to UserRatingMS via Rabbit MQ Asynchronously
- So we need to use Rabbit MQ in BookStoreWeb.

### Working Steps:

#### 1) Add the following dependency to pom.xml

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-amqp</artifactId>
</dependency>
```

#### 2) Add the following to application.properties

```
spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest
```

#### 3) Create the package called **com.coursecube.bookstoreweb.rabbitmq** and place the following classes.

```
***OrderFullInfo.java
***OrderInfo.java
***OrderItemInfo.java
***UserRatingInfo.java
```

```
class OrderInfo implements Serializable{
    private Integer orderId;
    private String orderDate;
    private String userId;
    private int totalQty;
    private double totalCost;
    private String status;
    ...
}
```



```
class OrderItemInfo implements Serializable {
    private Integer orderItemId;
    private Integer orderId;
    private Integer bookId;
    private int qty;
    private double cost;
    ...
}
class OrderFullInfo implements Serializable {
    private Order order;
    private List<OrderItem> itemList;
    ...
}
class UserRatingInfo implements Serializable{
    private Integer ratingId;
    private String userId;
    private Integer bookId;
    private double rating;
    private String review;
    ...
}
```

#### 4) Update the methods of BookStoreServiceImpl

```
@Autowired
private RabbitTemplate rabbitTemplate;
```

```
//Sending Order Message to RabbitMQ
```

```
OrderFullInfo orderFullInfo = new OrderFullInfo();
orderFullInfo.setOrder(orderInfo);
orderFullInfo.setItemList(itemList);
```

```
rabbitTemplate.convertAndSend("myorder.exchange","myorder.key",
orderFullInfo);
```

```
//Sending UserRating Message RabbitMQ
```

```
UserRatingInfo userRatingInfo=new UserRatingInfo(userRating.getUserId(),
userRating.getBookId(), userRating.getRating(), userRating.getReview());
```

```
rabbitTemplate.convertAndSend("myuser.ratings.exchange","myuser.ratings.ke
y", userRatingInfo);
```



## BookStoreWeb – V-1.2: Files required

|                                  |                             |
|----------------------------------|-----------------------------|
| 1. index.jsp                     | Same as V1.1                |
| 2. myheader.jsp                  | Same as V1.1                |
| 3. myfooter.jsp                  | Same as V1.1                |
| 4. showBooksList.jsp             | Same as V1.1                |
| 5. showBookInfo.jsp              | Same as V1.1                |
| 6. showCart.jsp                  | Same as V1.1                |
| 7. orderSuccess.jsp              | Same as V1.1                |
| 8. addRating.jsp                 | Same as V1.1                |
| 9. ratingSuccess.jsp             | Same as V1.1                |
| 10. ratingsList.jsp              | Same as V1.1                |
| 11. bookstore.css                | Same as V1.1                |
| 12. Book.java                    | Same as V1.1                |
| 13. BookInfo.java                | Same as V1.1                |
| 14. Order.java                   | Same as V1.1                |
| 15. OrderItem.java               | Same as V1.1                |
| 16. UserRating.java              | Same as V1.1                |
| 17. BookStoreController.java     | Same as V1.1                |
| 18. OrderFullInfo.java           | <b>Newly added in V.1.2</b> |
| 19. OrderInfo.java               | <b>Newly added in V.1.2</b> |
| 20. OrderItemInfo.java           | <b>Newly added in V.1.2</b> |
| 21. UserRatingInfo.java          | <b>Newly added in V.1.2</b> |
| 22. BookStoreService.java        | Same as V1.1                |
| 23. BookStoreServiceImpl.java    | <b>Updated in V1.2</b>      |
| 24. JLCBookStoreWebConfig.java   | Same as V1.1                |
| 25. BookStoreWebApplication.java | Same as V1.1                |
| <b>26.application.properties</b> | <b>Updated in V1.2</b>      |
| <b>27.pom.xml</b>                | <b>Updated in V1.2</b>      |

### 18.OrderFullInfo.java

```
package com.jlcindia.rabbitmq;
```

```
import java.io.Serializable;
```

```
import java.util.List;
```

```
/*
```

```
* @Author : Srinivas Dande
```



```
* @company : Java Learning Center
```

```
**/
```

```
public class OrderFullInfo implements Serializable {  
  
    private OrderInfo order;  
    private List<OrderItemInfo> itemsList;  
  
    //Constructors  
    //Setters and Getters  
}
```

### 19. OrderInfo.java

```
package com.jlcindia.rabbitmq;  
  
import java.io.Serializable;  
/*  
* @Author : Srinivas Dande  
* @company : Java Learning Center  
**/  
public class OrderInfo implements Serializable{  
  
    private Integer orderId;  
    private String orderDate;  
    private String userId;  
    private int totalQty;  
    private double totalCost;  
    private String status;  
  
    //Constructors  
    //Setters and Getters  
}
```

### 20. OrderItemInfo.java

```
package com.jlcindia.rabbitmq;  
  
import java.io.Serializable;  
/*  
* @Author : Srinivas Dande  
* @company : Java Learning Center  
**/  
public class OrderItemInfo implements Serializable {
```



```
private Integer orderItemId;  
private Integer orderId;  
private Integer bookId;  
private int qty;  
private double cost;
```

```
//Constructors  
//Setters and Getters
```

```
}
```

### **21.UserRatingInfo.java**

```
package com.jlcindia.rabbitmq;
```

```
import java.io.Serializable;
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @company : Java Learning Center
```

```
**/
```

```
public class UserRatingInfo implements Serializable{
```

```
private Integer ratingId;  
private String userId;  
private Integer bookId;  
private double rating;  
private String review;
```

```
//Constructors  
//Setters and Getters  
//toString
```

```
}
```

### **23. BookStoreServiceImpl.java**

```
package com.jlcindia.bookstoreweb;
```

```
import java.text.SimpleDateFormat;
```

```
import java.util.ArrayList;
```

```
import java.util.Calendar;
```

```
import java.util.Date;
```

```
import java.util.LinkedHashMap;
```

```
import java.util.List;
```

```
import java.util.Map;
```



```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.amqp.rabbit.core.RabbitTemplate;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestTemplate;

import com.jlcindia.rabbitmq.OrderFullInfo;
import com.jlcindia.rabbitmq.OrderInfo;
import com.jlcindia.rabbitmq.OrderItemInfo;
import com.jlcindia.rabbitmq.UserRatingInfo;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

@Service
public class BookStoreServiceImpl implements BookStoreService {

    static Logger log = LoggerFactory.getLogger(BookStoreServiceImpl.class);

    Map<Integer, Book> booksMap = new LinkedHashMap<>();

    @Autowired
    private RabbitTemplate rabbitTemplate;

    public List<String> getAuthorsList() {
        List<String> authorsList = new ArrayList<>();
        authorsList.add("All Authors");
        authorsList.add("Srinivas");
        authorsList.add("Vas");
        authorsList.add("Sri");
        return authorsList;
    }

    public List<String> getCategoryList() {
        List<String> catList = new ArrayList<>();
        catList.add("All Categories");
        catList.add("Web");
        catList.add("Spring");
        return catList;
    }
}
```



```
public List<Book> getMyBooks(String author, String category) {
    System.out.println("BookStoreServiceImpl - getBooks()");

    if (author == null || author.length() == 0) {
        author = "All Authors";
    }
    if (category == null || category.length() == 0) {
        category = "All Categories";
    }

    // Invoke BookSearchMS

    RestTemplate bookSearchRest = new RestTemplate();
    String endpoint = "http://localhost:8000/mybooks/" + author + "/" + category;

    List<Map> list = bookSearchRest.getForObject(endpoint, List.class);

    List<Book> bookList = new ArrayList<>();

    for (Map mymap : list) {
        Book mybook = convertMapToBook(mymap);
        bookList.add(mybook);
        booksMap.put(mybook.getBookId(), mybook);
    }

    return bookList;
}

public BookInfo getBookInfoByBookId(Integer bookId) {
    System.out.println("BookStoreServiceImpl - getBookInfoByBookId()");

    // Invoke BookSearchMS

    RestTemplate bookSearchRest = new RestTemplate();
    String endpoint = "http://localhost:8000/mybook/" + bookId;

    BookInfo bookInfo = bookSearchRest.getForObject(endpoint, BookInfo.class);

    return bookInfo;
}
```



```
public Book getBookById(Integer bookId) {
    System.out.println("BookStoreServiceImpl - getBookById()");
    System.out.println(bookId);
    Book mybook = booksMap.get(bookId);

    return mybook;
}

public void placeOrder(Map<Integer, Book> mycartMap) {

    // New Code
    System.out.println("---2.BookStoreServiceImpl--placeOrder()----");
    List<OrderItemInfo> itemList = new ArrayList<>();
    double totalPrice = 0.0;
    int totalQuantity = 0;

    for (Book mybook : mycartMap.values()) {
        Integer bookId = mybook.getId();

        // Invoke BookPrice MS
        RestTemplate bookPriceRest = new RestTemplate();
        String priceEndpoint = "http://localhost:9000/offeredPrice/" + bookId;
        double offerPrice = bookPriceRest.getForObject(priceEndpoint, Double.class);

        OrderItemInfo item = new OrderItemInfo(0, bookId, 1, offerPrice);
        itemList.add(item);

        totalPrice = totalPrice + offerPrice;
        totalQuantity = totalQuantity + 1;
    }

    Date today = Calendar.getInstance().getTime();
    SimpleDateFormat formatter = new SimpleDateFormat("dd-MM-yyyy hh:mm");
    String orderDate = formatter.format(today);
    System.out.println(orderDate);

    OrderInfo orderInfo = new OrderInfo(orderDate, "U-111", totalQuantity, totalPrice, "New");

    OrderFullInfo orderFullInfo = new OrderFullInfo();
    orderFullInfo.setOrder(orderInfo);
    orderFullInfo.setItemsList(itemList);
}
```



```
// Sending Order Message to RabbitMQ
rabbitTemplate.convertAndSend("myorder.exchange","myorder.key",
orderFullInfo);

System.out.println("Order Placed");
}

@Override
public List<Order> getMyOrders(String userId) {
// Invoke PlaceOrder MS
String orderEndpoint = "http://localhost:7000/myorders/U-123";
RestTemplate orderRest = new RestTemplate();

List<Order> myorders = orderRest.getForObject(orderEndpoint, List.class);
// You need to Implement Remaining
return myorders;
}

public void addUserRating(UserRating userRating) {
System.out.println("---2.BookStoreServiceImpl--addUserRating()----");

//Sending UserRating Message RabbitMQ
UserRatingInfo userRatingInfo=new UserRatingInfo(userRating.getUserId(),
userRating.getBookId(), userRating.getRating(), userRating.getReview());

rabbitTemplate.convertAndSend("myuser.ratings.exchange","myuser.ratings.key",
userRatingInfo);

System.out.println("Rating Added");
}

public List<UserRating> getMyRatings(String userId) {
List<UserRating> ratingsList = new ArrayList<>();

String ratingEndpoint = "http://localhost:6500/userRatings/" + userId;
RestTemplate ratingRest = new RestTemplate();
List<Map> mymap = ratingRest.getForObject(ratingEndpoint, List.class);

for (Map map : mymap) {
UserRating urtaing = convertMapToUserRating(map);
ratingsList.add(urtaing);
System.out.println(map);
}
}
```



```
return ratingsList;

}

private UserRating convertMapToUserRating(Map map) {
    UserRating rating = new UserRating();
    rating.setRatingId(new Integer(map.get("ratingId").toString()));
    rating.setUserId(map.get("userId").toString());
    rating.setBookId(new Integer(map.get("bookId").toString()));
    rating.setRating(new Double(map.get("rating").toString()));
    rating.setReview(map.get("review").toString());

    return rating;
}

private Book convertMapToBook(Map map) {
    Book mybook = new Book();
    mybook.setBookId(Integer.parseInt(map.get("bookId").toString()));
    mybook.setBookName((map.get("bookName").toString()));
    mybook.setAuthor((map.get("author").toString()));
    mybook.setPublications(map.get("publications").toString());
    mybook.setCategory(map.get("category").toString());
    return mybook;
}
}
```

## **26. application.properties**

```
spring.application.name=MyBookStoreWeb

server.port=5000

spring.boot.admin.client.url=http://localhost:5555

spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest

management.endpoints.web.exposure.include=*
management.endpoint.health.show-details=always

logging.level.root=INFO
```



```
logging.pattern.console=%-5level %logger{36} - %msg%n
```

```
# INFO ENDPOINT CONFIGURATION
```

```
info.app.name=@project.name@
```

```
info.app.description=@project.description@
```

```
info.app.version=@project.version@
```

```
info.app.encoding=@project.build.sourceEncoding@
```

```
info.app.java.version=@java.version@
```

```
info.app.developer=Srinivas Dande
```

```
info.app.company=Java Learning Center
```

## 27.pom.xml

```
<project .....">
```

```
<modelVersion>4.0.0</modelVersion>
```

```
<groupId>com.jlcindia.bookstoreweb</groupId>
```

```
<artifactId>BookStoreWeb</artifactId>
```

```
<version>1.2</version>
```

```
<packaging>jar</packaging>
```

```
<name>MyBookStoreWeb</name>
```

```
<url>https://www.jlcindia.com</url>
```

```
<description>This is BookStoreWeb developed by Srinivas Dande</description>
```

```
<parent>
```

```
<groupId>org.springframework.boot</groupId>
```

```
<artifactId>spring-boot-starter-parent</artifactId>
```

```
<version>2.5.3</version>
```

```
</parent>
```

```
<dependencies>
```

```
<dependency>
```

```
<groupId>org.springframework.boot</groupId>
```

```
<artifactId>spring-boot-starter-amqp</artifactId>
```

```
</dependency>
```

```
<dependency>
```

```
<groupId>de.codecentric</groupId>
```

```
<artifactId>spring-boot-admin-starter-client</artifactId>
```

```
<version>2.4.3</version>
```

```
</dependency>
```



```
<dependency>  
<groupId>org.springframework.boot</groupId>  
<artifactId>spring-boot-starter-actuator</artifactId>  
</dependency>
```

```
<dependency>  
<groupId>org.springframework.boot</groupId>  
<artifactId>spring-boot-starter-web</artifactId>  
</dependency>
```

```
<dependency>  
<groupId>org.apache.tomcat.embed</groupId>  
<artifactId>tomcat-embed-jasper</artifactId>  
</dependency>
```

```
<dependency>  
<groupId>jstl</groupId>  
<artifactId>jstl</artifactId>  
<version>1.2</version>  
</dependency>
```

```
<dependency>  
<groupId>taglibs</groupId>  
<artifactId>standard</artifactId>  
<version>1.1.2</version>  
</dependency>
```

```
<dependency>  
<groupId>org.webjars</groupId>  
<artifactId>bootstrap</artifactId>  
<version>4.3.1</version>  
</dependency>
```

```
<dependency>  
<groupId>org.webjars</groupId>  
<artifactId>jquery</artifactId>  
<version>3.4.1</version>  
</dependency>
```

```
<dependency>  
<groupId>org.springframework.boot</groupId>  
<artifactId>spring-boot-devtools</artifactId>  
</dependency>
```



```
</dependencies>

<build>
<finalName>MyBookStoreWeb</finalName>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>

</project>
```

| MicroService Name | Rest Endpoints                                    | Used By        |
|-------------------|---------------------------------------------------|----------------|
| MyBookPriceMS     | http://localhost:9000/bookPrice/{bookId}          | MyBookSearchMS |
| MyBookPriceMS     | http://localhost:9000/offeredPrice/{bookId}       | MyBookStoreWeb |
| MyBookSearchMS    | http://localhost:8000/mybooks/{author}/{category} | MyBookStoreWeb |
| MyBookSearchMS    | http://localhost:8000/mybook/{bookId}             | MyBookStoreWeb |
| MyPlaceOrderMS    | http://localhost:7000/myorders/{userId}           | MyBookStoreWeb |
| MyUserRatingMS    | http://localhost:6500/userRatings/{userId}        | MyBookStoreWeb |